# Off-Grid Emergency Messaging Using a Self-Healing ESP32-LoRa Mesh

**B. Haasini D. Hemanth Madhav G. Shriivatsa M. Anand Sai V. Sucheth**
ammulubobba@gmail.com hemanthmadhavdone@gmail.com shriivatsag@gmail.com
anandsai.modugul@gmail.com veerasucheth@gmail.com

**Abstract—** This paper presents a four-node, off-grid emergency messaging prototype built with ESP32 microcontrollers and SX1278 LoRa radio modules. The system is intended to support communication when cellular networks, Wi-Fi, and internet access are unavailable or disrupted. Each handheld node provides an OLED-based interface navigated with a rotary encoder, allowing users to send emergency presets, direct messages, and broadcasts. The firmware includes packet forwarding with a time-to-live (TTL), duplicate-packet suppression, neighbor activity monitoring, delivery acknowledgements, read acknowledgements, message history, and relay-path visibility. The code also includes an optional headless relay build. The firmware currently demonstrates acknowledgement and forwarding logic; an automatic retransmission/retry mechanism for unacknowledged messages was not identified in the supplied sketch and should not be claimed unless added and tested. Performance claims, including successful recovery after a node failure, must be supported by measured experiments. Index Terms—ESP32, LoRa, SX1278, emergency messaging, wireless mesh, packet forwarding, acknowledgements, OLED, rotary encoder.

## I. INTRODUCTION

During natural disasters, rescue operations, and work in remote areas, cellular towers, Wi-Fi, and internet services may be unavailable, damaged, or overloaded. People and responders may then lack a practical way to exchange short emergency messages. Satellite phones can be expensive and walkie-talkies commonly depend on direct radio links; a simple point-to-point LoRa link also does not by itself provide multi-hop forwarding or end-to-end delivery status.

This project addresses these limitations with a low-cost ESP32–LoRa prototype. Messages contain source and destination identifiers and are forwarded by nodes that receive them, subject to packet lifetime and duplicate suppression. The user interface uses a 128×64 SSD1306 OLED and an HW-040 rotary encoder so a user can navigate menus and send predefined emergency messages or custom text without a phone or internet connection.

## II. BACKGROUND AND RELATED WORK

### *A. LoRa for Infrastructure-Independent Messaging*

LoRa is a low-power radio technology suitable for exchanging small packets over local wireless links. Actual range depends on antennas, radio configuration, terrain, obstructions, interference, and local regulations; therefore, the range of this prototype should be established by testing rather than assumed.

In a mesh-style design, nodes can forward packets for other nodes so that a destination does not always need to be within direct radio range of the sender. Forwarding must be controlled to avoid repeated circulation of the same packet. The supplied firmware uses a packet TTL, sequence information, a recently-seen cache, and a recorded relay path.

### *B. Emergency Messaging and User Interfaces*

Emergency communication interfaces should minimize the number of actions required under stress. This prototype provides an OLED menu navigated with a rotary encoder, predefined emergency message options, a custom-message path, a broadcast channel, an inbox, an outbox, and previous chat threads.

### *C. Multi-Hop Forwarding and Network Resilience*

Multi-hop forwarding can extend communication beyond a single direct link, but forwarding alone does not guarantee that every message will arrive. This firmware tracks neighboring nodes based on received packets, marks a neighbor offline after a configured timeout, and forwards data and acknowledgement packets using a flood-forwarding approach with duplicate suppression and TTL limits.

### *D. Delivery and Read Acknowledgements*

For addressed unicast messages, the destination automatically sends a delivery acknowledgement when the DATA packet arrives. A separate read acknowledgement is sent when the user deliberately opens a message or thread. These are different states: delivered means the destination node received the packet, while read means the recipient opened it in the interface.

### *E. Design Challenges*

Important implementation challenges include radio-parameter consistency across nodes, avoiding forwarding loops, detecting inactive neighbors, communicating delivery status, and keeping the interface usable on a small display. The supplied code explicitly configures the LoRa radio settings and enables CRC. The system should be evaluated under controlled node/link failures before claiming complete self-healing or guaranteed delivery.

## III. PROBLEM DEFINITION

During natural disasters, in remote areas, and in rescue operations, cellular towers, Wi-Fi, and the internet may be damaged, overloaded, or unavailable. Responders and civilians then have no reliable way to send alerts or coordinate. Satellite phones may be expensive and not widely available; walkie-talkies are commonly point-to-point and offer limited relaying; and basic LoRa links can fail when a fixed communication path is broken, often without informing the sender whether a message arrived.

The project aims to provide a low-cost, infrastructure-independent communication system that can relay messages across multiple hops, monitor node availability, and display delivery/read acknowledgements. It also provides a simple OLED-and-rotary-encoder interface for emergency presets and messaging. The current firmware includes acknowledgement packets and neighbor-status detection; automatic retries of unacknowledged messages were not found in the supplied code, so retries remain a requirement for future implementation unless verified in another code version.

## IV. PROPOSED SYSTEM

The proposed prototype consists of four ESP32-based LoRa nodes. Each user-facing node uses an SX1278 radio configured at 433 MHz, a 128×64 SSD1306 I2C OLED, and an HW-040 rotary encoder. A node is assigned a unique ID using the NODE_ID definition (1–4 for handheld nodes). All nodes must use identical LoRa settings. Each fixed-size packet includes a packet type (data, hello, delivery acknowledgement, or read acknowledgement), source, destination, sequence number, TTL, a reference sequence number for acknowledgements, a sender timestamp, relay-path metadata, and a 32-byte payload. Nodes suppress duplicates and forward packets that are not addressed to them while the TTL permits. The code also supports an optional headless relay build using RELAY_NODE_MODE, with a non-user node ID such as 90 or 91.

### *A. Example Use Scenario*

A responder in an area without internet access selects an emergency preset or writes a short message using the rotary encoder and OLED menu. The originating node transmits the packet over LoRa. Other nodes that hear the packet can forward it, allowing multi-hop delivery when the topology permits. The destination returns a delivery acknowledgement, and opening the message can generate a read acknowledgement. The sender can inspect message status and the recorded relay path where available.

If a node stops communicating, the firmware can mark it offline after the configured timeout. Flood-forwarding may allow packets to travel through another available path, but the team must demonstrate this behavior experimentally before claiming that every failure automatically results in successful rerouting.

### *B. SYSTEM ARCHITECTURE*

User Node: ESP32 microcontroller, SX1278 LoRa module, SSD1306 OLED display, and HW-040 rotary encoder. The user interface includes Emergency, Send Message, Broadcast, Inbox, Outbox, Previous Chats, and Settings menus. The same sketch is used across handheld nodes; only NODE_ID is changed for each device.

LoRa Radio: SX1278 connected to the ESP32 through SPI. The firmware uses 433 MHz, spreading factor 9, 125 kHz bandwidth, coding rate 4/5, sync word 0x12,

transmit-power setting 20 dBm, and CRC. These settings must match on all nodes and must be checked against the specific module and applicable radio regulations.

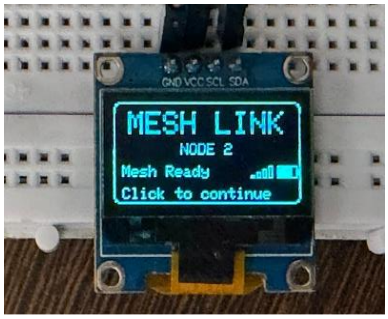



Fig. 1. SSD1306 OLED user interface

OLED and Rotary Encoder Interface: The OLED presents menus, incoming-message notifications, inbox/outbox entries, node status, delivery/read indicators, and settings. Rotating the encoder changes the selection or the character being entered; a short press selects an item or picks a character. A long press acts as Back on every screen. On an incoming-message popup, a long press dismisses the message and sends a read acknowledgement.

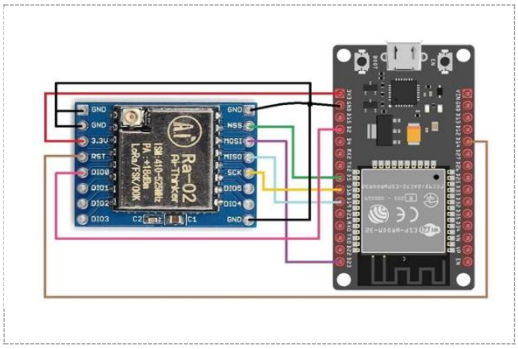



Fig. 2. SX1278 LoRa module and ESP32 SPI wiring

Emergency Messaging: The main menu begins with an EMERGENCY entry that opens a grid of six presets (HELP, CALL ME, URGENT, SOS, MEDICAL, and FIRE), which are sent as a broadcast to all nodes. Choosing Send Message instead lets the user pick a destination node or Broadcast, and then a send menu with Emergency, Medical, Fire, Need Backup, Hostage Found, and Custom Message. Custom text (up to 31 characters) is entered by turning the encoder through a character set (A–Z, 0–9, space, a delete symbol, and a send symbol). These are software menu choices, not a separate physical panic button.

## V. IMPLEMENTATION

### *A. User Node Hardware*

Each handheld node uses an ESP32 development board, SX1278 LoRa module, SSD1306 128×64 I2C OLED, and HW-040 rotary encoder. The supplied firmware defines the OLED I2C pins, rotary encoder pins, and LoRa SPI pins shown in the connection tables. No GPS module or dedicated panic button is used by the supplied mesh firmware.

#### *1) OLED to ESP32 Connections:*

TABLE I
HARDWARE CONNECTIONS

| OLED Pin | ESP32 Pin |
|---|---|
| VCC | 3.3V (check your module specification) |
| GND | GND |
| SDA | GPIO21 |
| SCL | GPIO22 |

#### *2) LoRa to ESP32 Connections:*

TABLE II
SX1278 LORA TO ESP32 CONNECTIONS

| LoRa Pin | ESP32 Pin |
|---|---|
| VCC | 3.3V |
| GND | GND |
| SCK | GPIO18 |
| MISO | GPIO19 |
| MOSI | GPIO23 |
| NSS / CS | GPIO5 |
| RST | GPIO14 |
| DIO0 | GPIO2 |

#### *3) Rotary Encoder to ESP32 Connections:*

TABLE III
ROTARY ENCODER TO ESP32 CONNECTIONS

| Rotary Encoder Pin | ESP32 Pin |
|---|---|
| CLK | GPIO27 |
| DT | GPIO26 |
| SW | GPIO25 |
| VCC | 3.3V (module-dependent; check HW-040 board) |
| GND | GND |

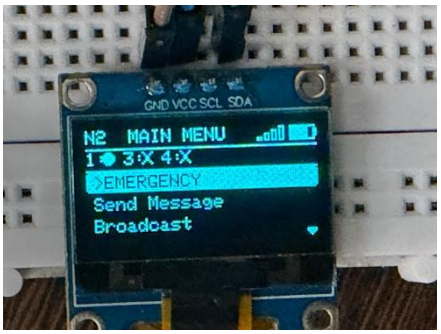



Fig. 3. OLED and rotary encoder interface

Relay and Neighbor Monitoring: Each node records recently heard neighbors and signal strength (RSSI). A node is marked offline if no packet is heard within the configured 15-second timeout. When RELAY_NODE_MODE is enabled, the device runs as a headless forwarder without OLED or encoder UI and can blink GPIO13 when it relays a packet. Any received packet (HELLO heartbeat, data, or acknowledgement) counts as proof that its source node is alive. Each node broadcasts a HELLO packet every 5 seconds, with a random start offset so that nodes do not transmit in lockstep. The neighbor table holds up to 10 entries. A short "Node N Reconnected" popup is shown when an offline node is heard again.

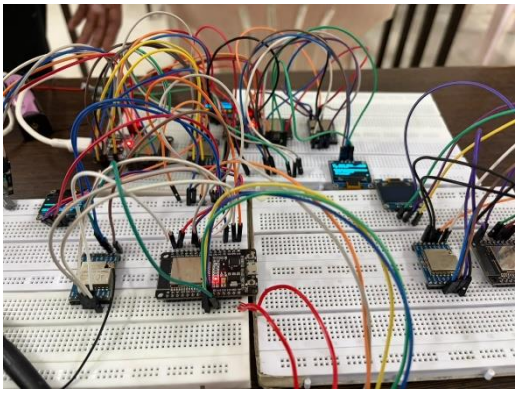


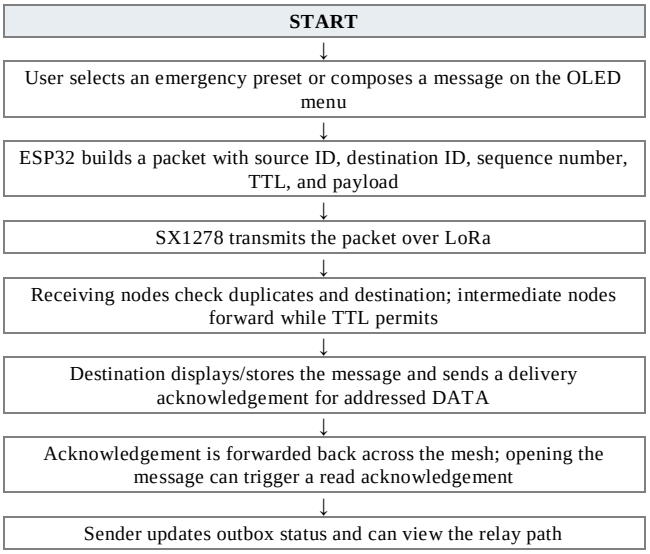
Fig. 4. Four-node ESP32–LoRa mesh prototype

Message and Status Handling: Incoming messages are queued for on-screen display and recorded in the inbox/chat history. Outgoing messages are stored in the outbox with status tracking. Unicast messages can progress from sent to delivered to read; broadcast messages do not have a single recipient and therefore do not use per-recipient delivery/read ticks.

Time and Settings: The Settings menu lists Node ID, Brightness, LoRa Frequency, Set Clock, Factory Reset, and About. In the current firmware, Node ID and LoRa Frequency are information screens only (they are fixed in the code before flashing), and Brightness is a placeholder that is not yet connected to the display contrast. Factory Reset clears the stored chat history (inbox, per-node chats, outbox, and broadcast log). Set Clock lets the user enter the hour and minute manually; the software clock is based on millis() and resets to unset after reboot because the design has no RTC chip or network time source. Screens that show a time fall back to a relative "Xs ago" display when a clock has not been set. The battery indicator on the display currently shows a fixed placeholder value (82%) and is not measured from hardware.

The architecture combines short-message exchange, flood-forwarding, duplicate suppression, neighbor-status monitoring, and delivery/read acknowledgements. The OLED and rotary encoder make the system usable without a smartphone. This report describes the features visible in the supplied firmware; measured reliability and recovery performance must be added after testing.

### *B. Packet and Message Flow*




### *C. Firmware Working Steps*

1. Configure a unique NODE_ID for each handheld node (1, 2, 3, or 4).
2. Wire the OLED, rotary encoder, and SX1278 radio according to the pin tables.
3. Configure identical LoRa frequency, spreading factor, bandwidth, coding rate, sync word, and transmit power on every node.
4. Navigate the OLED menus using the rotary encoder and send a direct message, broadcast, or emergency preset.
5. Receiving nodes validate packet size, update neighbor activity, and suppress duplicate packets.
6. Nodes forward eligible packets while TTL remains; addressed destinations return delivery acknowledgements.
7. Opening a message or thread can generate a read acknowledgement, and the outbox updates status.
8. Test node-offline detection and alternate-path delivery with repeatable controlled experiments.

### *D. Optional Relay Build*

The firmware includes RELAY_NODE_MODE. When set to 1 and given an ID outside 1–4 (for example 90), the board operates as a headless relay and does not initialize the OLED or rotary encoder UI. GPIO13 is used as a relay-activity LED. This is a compile-time mode; confirm whether the actual four-node demonstration uses four handheld nodes or includes a board flashed in relay mode.

Relay mode working steps:

1. Set RELAY_NODE_MODE to 1 and choose a unique relay NODE_ID outside 1–4.
2. Connect the ESP32 to the SX1278 using the LoRa SPI pins in Table II.
3. Flash the same sketch with the relay-mode setting and matching LoRa parameters.
4. The relay listens for packets, suppresses duplicates, forwards eligible packets, and flashes GPIO13 on relay activity.

### *E. Software Implementation*

The firmware is written for the ESP32 in Arduino IDE. It includes SPI and LoRa libraries for SX1278 radio communication, Wire and Adafruit SSD1306/Adafruit GFX for the I2C OLED, and standard C string/stdio functions for packet and display formatting. The rotary encoder is read using GPIO inputs and an interrupt on the CLK pin. The sketch configures radio parameters explicitly and enables CRC. No GPS or TinyGPS++ code is used in the supplied firmware.

Packet format: Every transmission is a fixed-size Packet structure with the fields type, source, destination, seq, ttl, refSeq, epoch, hopCount, path[6], and payload[32]. The type field takes one of four values: PKT_DATA, PKT_HELLO, PKT_ACK_DELIVERED, and PKT_ACK_READ. Destination 0xFF is the broadcast address. The refSeq field is used only by acknowledgements and names the sequence number of the data packet being acknowledged. The epoch field carries the sender's wall-clock time (0 if its clock is not set). Receivers discard any packet whose length does not match the structure size.

Forwarding logic: Data and acknowledgement packets start with a TTL of 5 and HELLO packets with a TTL of 1. A node that receives a packet not addressed to it checks a 20-entry recently-seen cache of (source, sequence) pairs, drops duplicates, decrements the TTL, and rebroadcasts the packet if the TTL has not reached zero. For data packets the forwarding node also appends its own ID to the path array (up to 6 relays) and refuses to forward a packet whose path already contains its ID, which acts as an additional loop guard. HELLO packets are never forwarded.

Acknowledgements: When a data packet addressed to a node arrives, the radio layer immediately returns a unicast PKT_ACK_DELIVERED, copying the data packet's hop count and relay path into the acknowledgement unchanged. The sender therefore learns the path its message took, not the path the acknowledgement took back. Broadcast messages are not acknowledged, to avoid every listening node replying. PKT_ACK_READ is sent only when the user deliberately opens a message (dismissing the popup with a long press, opening it from the Inbox, or opening its chat thread); a popup that merely times out does not count as read. A 16-entry cache prevents the same read acknowledgement from being sent twice, and a message's status can only move forward (sent, then delivered, then read) even if acknowledgements arrive out of order.

Message storage and display: Incoming messages enter an 8-slot circular display queue and are shown one after another on the incoming-message screen, each for about 4 seconds. If the queue overflows, the oldest unshown message is dropped so that the newest one is never lost. All messages are also logged (until reset or reboot) in a 20-entry inbox, a 10-message thread for each of the other nodes, a 20-entry outbox, and a 16-entry shared broadcast channel. Outgoing messages in a chat thread show one tick for sent, two ticks for delivered, and two ticks on a solid block for read. The Outbox lists each sent message with a status tag S, D, or R and, once the delivered acknowledgement arrives, the relay path in the form 1>3>4 (sender, relays, destination). A "..." marker is shown while a message to a non-neighbor node is still awaiting its acknowledgement.

Offline handling: A destination is treated as offline if it is not an active neighbor, meaning no packet from it has been heard in the last 15 seconds. If the user sends to an offline node, an "OFFLINE – not delivered, node unreachable" warning is shown for 2 seconds, and the message is still transmitted so that it can be relayed if a path exists.

### *F. Testing and Validation*

Testing should cover direct messaging, multi-hop forwarding, broadcast reception, delivery acknowledgements, read acknowledgements, duplicate suppression, neighbor online/offline transitions, and relay-path reporting. The code sets a 15-second neighbor-offline timeout. The test report should record node positions, antenna orientation, radio settings, number of messages sent/received, and whether messages were delivered through an alternate path.

No verified range, packet-delivery percentage, or recovery-time measurements were supplied with the firmware. Do not state that the system achieves a particular distance or has proven reliable long-range operation until these values have been measured and recorded.

The OLED/rotary interface should be tested for menu navigation, emergency preset selection, custom-message entry, inbox/outbox display, and status updates. Node-failure tests should distinguish offline detection from successful delivery over an alternative route.

The supplied code contains delivery and read acknowledgement packet types, but an explicit retry/resend loop for unacknowledged messages was not identified. If retries are a required feature, implement them and test them before describing them as complete.

## VI. RESULTS AND DISCUSSION

The supplied firmware supports the core software paths for message sending, forwarding, duplicate suppression, neighbor activity tracking, and delivery/read acknowledgements. This is a description of code features, not a claim that all functions have been validated in field conditions.

Results should be reported only after testing the physical four-node setup. At minimum, record successful and failed message counts, the number of forwarding hops, acknowledgement latency, behavior after a node is powered off, and whether new messages reach the destination through an alternative path.

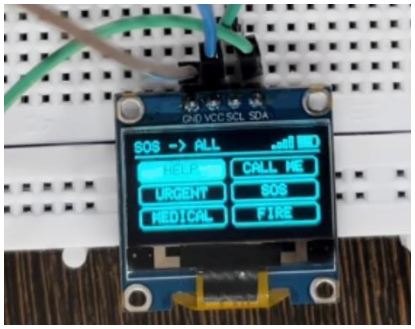



Fig. 5. OLED emergency-message screen

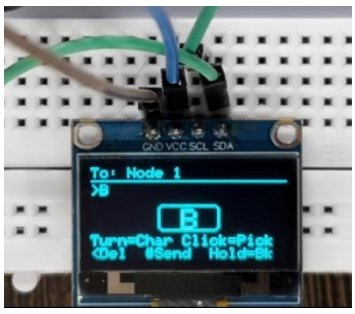



Fig. 6. OLED emergency-Custom message screen

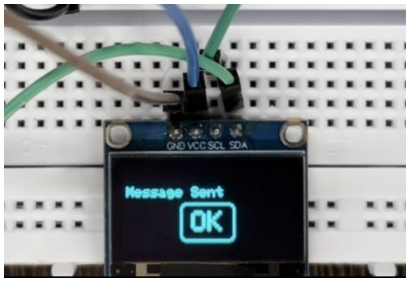



Fig. 7. Screen of Transmitter after sending a message

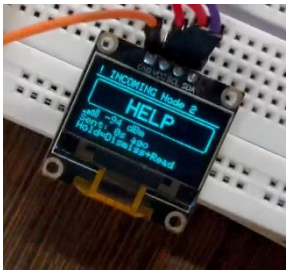



Fig. 8. Screen of Receiver after receiving a message

### *A. Applications*

Military Rescue Operations: In a military or search-and-rescue context, operations may be carried out in remote territory where radio communications are unreliable or non-existent. The system allows short preset alerts such as SOS, MEDICAL, or NEED BACKUP to be broadcast to nearby team members and relayed onward, without any fixed infrastructure. The current firmware does not report GPS location.

Disaster response and rescue coordination in areas where conventional network infrastructure is unavailable.

Remote work sites where teams need to exchange short status or emergency messages.

Temporary field operations, outdoor events, and expeditions where local off-grid messaging is useful.

Training and research on embedded systems, low-power radio links, and multi-hop networking.

Small teams coordinating across a limited area without relying on cellular or internet connectivity.

The system is a prototype and should not be relied upon as the sole communication method in a real emergency.

### *B. Limitations*

The current sketch does not include GPS location reporting, a web map, deep-sleep battery management, or a physical panic button. Its software clock is not retained across power cycles, and the battery indicator and brightness setting are placeholders. Delivery acknowledgements are implemented, but automatic message retries were not identified; a message to a node that is out of reach is transmitted once and is not resent. The node count is fixed at four handheld nodes (IDs 1–4), messages are limited to 31 characters, and the mesh uses simple flooding with a TTL of 5 and a 20-entry duplicate cache, so heavy traffic could cause old duplicate-cache entries to be overwritten and airtime contention. Messages are sent unencrypted. Network recovery and range must be measured under controlled conditions.

## VII. CONCLUSION

This project presents an ESP32–SX1278 off-grid messaging prototype with four configurable handheld node identities, an OLED/rotary-encoder interface, multi-hop flood-forwarding, duplicate suppression, neighbor-status monitoring, and delivery/read acknowledgements. These features address several limitations of simple point-to-point LoRa messaging.

The firmware also provides an optional headless relay mode and tracks the relay path for data packets. However, acknowledgement support should not be described as automatic retry, and neighbor-offline detection alone does not prove successful self-healing. Those claims require additional implementation or experimental evidence.

The next stage is to test the complete four-node setup, document the actual wiring and radio environment, measure delivery and recovery performance, and add retry logic if required by the project specification.

## VIII. FUTURE SCOPE

Future work includes implementing bounded retransmission with timeout/backoff for messages without acknowledgements, adding explicit route-quality or route-selection logic if needed, testing alternate-path delivery after node/link failure, and measuring packet-delivery ratio and recovery time. Further work may include persistent real-time clock hardware, message authentication/encryption, improved power management, and more extensive outdoor range testing. These are future enhancements, not claims about the current code.

## IX. REFERENCES


[1] M. Girolami, F. Mavilia, A. Berton, G. Marrocco, and G. M. Bianco, “An Experimental Dataset for Search and Rescue Operations in Avalanche Scenarios Based on LoRa Technology,” IEEE Access, 2024.

[2] Brincat, F. Busacca, L. Galluccio, J. S. Mertens, A. Musumeci, S. Palazzo, and A. Panebianco, “An Integrated Acoustic/LoRa System for Transmission of Multimedia Sensor Data over an Internet of Underwater Things,” 2020.

[3] Z. Li, J. Dai, Y. Luan, N. Sun, and L. Du, “LR-MPIBS: A LoRa-Based Maritime Position-Indicating Beacon System,” Applied Sciences, vol. 14, no. 3, 2024.

[4] U. Noreen, A. Bounceur, and L. Clavier, “A Study of LoRa Low Power and Wide Area Network Technology,” 2017.

[5] R. Sharma, R. Raghuwanshi, T. Chandak, and D. Ramdasi, “LoRa-based IoT System for Emergency Assistance and Safety in Mountaineering,” International Journal of Safety and Security Engineering, vol. 13, no. 3, pp. 491–500, 2023.

[6] Niwangune, S. Pujari, T. Khandage, and S. P. Hon, “Safe Connect: A LoRa, GSM, and GPS-Based Women’s Emergency Alert System,” IRJET, vol. 12, no. 7, 2025.

[7] N. Hashim, F. Idris, T. N. A. T. Ab Aziz, S. H. Johari, R. M. Nor, and N. A. Wahab, “Location Tracking Using LoRa,” International Journal of Electrical and Computer Engineering, vol. 11, no. 4, pp. 3123–3128, 2021.

[8] N. M. Obiri and H. Shikunzi, “LoRa-WAN Connectivity and Range Evaluation in a Rural Setting,” International Journal of Computer Applications, vol. 185, no. 3, 2023.

[9] B. Jones, A. Tang, and C. Neustaedter, “Remote Communication in Wilderness Search and Rescue: Implications for the Design of Emergency Distributed Collaboration Tools,” Proceedings of the ACM on Human-Computer Interaction, vol. 4, 2020.

[10] R. B. Santelices et al., “Tracking Device System Using LoRaWAN for Motorized and Non-Motorized Banca,” Open Access Research Journal of Multidisciplinary Studies, 2023.